



合肥师范学院
HEFEI NORMAL UNIVERSITY

实验报告

课程名称: web 程序设计

学院: 计算机与人工智能学院

专业: 计算机科学与技术

学号: 2410211124

姓名: 朱品赞

指导教师: 张由明

2025 年 12 月 27 日

实验五 日程管理系统综合实验

一、实验目的

- 1. 掌握 Servlet 反射路由机制，实现动态 URL 到方法的映射
- 2. 理解 MVC 分层架构，学会使用 Controller、Service、DAO 三层设计
- 3. 综合应用 JDBC、会话、过滤器等技术，实现完整的用户认证和权限控制
- 4. 学会使用通用 DAO 模式简化数据访问层代码，提高代码复用性
- 5. 理解企业级 Java Web 应用的整体设计和实现方式

二、实验内容及步骤

2.1 项目概述

日程管理系统（Schedule Management System）是一个完整的 Java Web 应用，采用传统 Servlet 架构实现，集成了 Servlet、JDBC、会话管理、过滤器等多项 Java Web 核心技术。

项目功能

功能	说明
用户注册	创建新用户账号，密码 MD5 加密存储
用户登录	身份验证，创建会话，存储用户信息
用户退出	销毁会话，清空用户信息
添加日程	创建新的待办事项
查看日程	显示用户的全部日程列表
更新日程	修改日程标题和完成状态
删除日程	删除不需要的日程记录

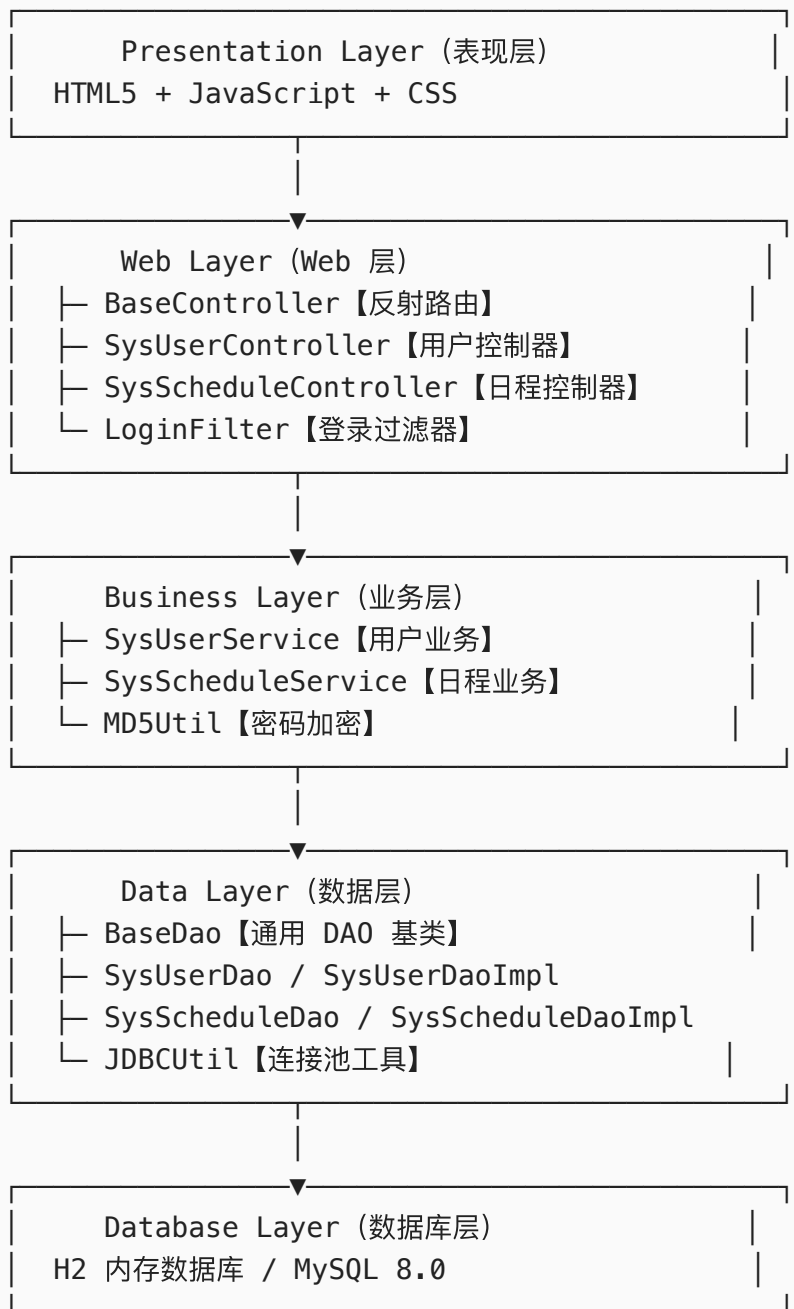
技术栈

技术	版本/说明
Java	Java 8
Servlet	3.1
数据库	H2（内存） / MySQL 8.0
连接池	Alibaba Druid

技术	版本/说明
ORM 工具	Apache Commons DbUtils
构建工具	Maven 3.9.12
应用服务器	Tomcat 7.0.47
前端	HTML5 + JavaScript + CSS

2.2 项目架构设计

分层架构



2.3 核心实现详解

BaseController - 反射路由机制

```
public abstract class BaseController extends HttpServlet {
    protected SysUserService sysUserService = new SysUserServiceImpl();
    protected SysScheduleService sysScheduleService = new
SysScheduleServiceImpl();

    @Override
    protected void service(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {
        // 设置字符编码
        request.setCharacterEncoding("UTF-8");
        response.setCharacterEncoding("UTF-8");
        response.setContentType("text/html;charset=UTF-8");

        // 获取请求的方法名
        String methodName = request.getParameter("method");

        if (methodName == null || methodName.isEmpty()) {
            response.getWriter().write("method 参数不能为空");
            return;
        }

        try {
            // 使用反射获取对应的方法
            Method method = this.getClass().getMethod(methodName,
                HttpServletRequest.class, HttpServletResponse.class);

            // 调用方法
            method.invoke(this, request, response);
        } catch (NoSuchMethodException e) {
            response.getWriter().write("方法 " + methodName + " 不存在");
        } catch (Exception e) {
            e.printStackTrace();
            response.getWriter().write("执行出错: " + e.getMessage());
        }
    }
}
```

反射路由工作原理：

1. 客户端请求 /user/login?method=login
↓
2. BaseController.service() 拦截请求
 - ├ 获取 method 参数值: login
 - ├ 使用反射: this.getClass().getMethod("login", ...)
 - ├ 反射找到 SysUserController.login() 方法
 - └ 调用 method.invoke(this, request, response)↓
3. 执行对应的业务逻辑

用户认证流程

Step 1: 用户注册

```
public void register(HttpServletRequest req, HttpServletResponse resp)
throws IOException {
    String username = req.getParameter("username");
    String password = req.getParameter("password");

    // 调用业务层检查用户名是否存在
    SysUser existUser = sysUserService.findByUsername(username);

    if (existUser != null) {
        resp.getWriter().write("用户名已存在");
        return;
    }

    // 创建用户对象
    SysUser sysUser = new SysUser();
    sysUser.setUsername(username);
    // 密码使用 MD5 加密
    sysUser.setUserPwd(MD5Util.encrypt(password));

    // 调用业务层保存用户
    boolean result = sysUserService.addUser(sysUser);

    if (result) {
        resp.getWriter().write("注册成功");
    } else {
        resp.getWriter().write("注册失败");
    }
}
```

Step 2: 用户登录

```

public void login(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    String username = req.getParameter("username");
    String password = req.getParameter("password");

    // 调用业务层进行身份验证
    SysUser sysUser = sysUserService.login(username, password);

    if (sysUser != null) {
        // 身份验证成功，创建会话
        HttpSession session = req.getSession();
        session.setAttribute("loginUser", sysUser);

        // 重定向到日程管理页面
        resp.sendRedirect(req.getContextPath() + "/showSchedule.html");
    } else {
        // 身份验证失败
        resp.sendRedirect(req.getContextPath() + "/loginError.html");
    }
}

```

Step 3: 用户退出

```

public void logout(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    // 获取会话并销毁
    HttpSession session = req.getSession();
    session.invalidate();

    // 重定向到登录页面
    resp.sendRedirect(req.getContextPath() + "/login.html");
}

```

日程管理 CRUD 操作

新增日程：

```

public void add(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    // 从会话获取当前用户
    HttpSession session = req.getSession();
    SysUser loginUser = (SysUser) session.getAttribute("loginUser");

    if (loginUser == null) {

```

```

        resp.sendRedirect(req.getContextPath() + "/login.html");
        return;
    }

    String title = req.getParameter("title");

    SysSchedule schedule = new SysSchedule();
    schedule.setUid(loginUser.getUid());
    schedule.setTitle(title);
    schedule.setCompleted(0);

    boolean result = sysScheduleService.addSchedule(schedule);

    if (result) {
        resp.getWriter().write("新增成功");
    } else {
        resp.getWriter().write("新增失败");
    }
}

```

查询日程列表：

```

public void list(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    HttpSession session = req.getSession();
    SysUser loginUser = (SysUser) session.getAttribute("loginUser");

    if (loginUser == null) {
        resp.getWriter().write("[]");
        return;
    }

    // 查询该用户的全部日程
    List<SysSchedule> scheduleList =
sysScheduleService.findByUid(loginUser.getUid());

    // 转换为 JSON 返回
    String json = JSON.toJSONString(scheduleList);
    resp.getWriter().write(json);
}

```

更新日程状态：

```

public void update(HttpServletRequest req, HttpServletResponse resp) throws
IOException {

```

```

Integer sid = Integer.parseInt(req.getParameter("sid"));
String title = req.getParameter("title");
Integer completed = Integer.parseInt(req.getParameter("completed"));

SysSchedule schedule = new SysSchedule();
schedule.setSid(sid);
schedule.setTitle(title);
schedule.setCompleted(completed);

boolean result = sysScheduleService.updateSchedule(schedule);

resp.getWriter().write(result ? "更新成功" : "更新失败");
}

```

删除日程：

```

public void delete(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    Integer sid = Integer.parseInt(req.getParameter("sid"));

    boolean result = sysScheduleService.deleteSchedule(sid);

    resp.getWriter().write(result ? "删除成功" : "删除失败");
}

```

通用 DAO 模式

BaseDao - 通用基类：

```

public abstract class BaseDao {
    protected QueryRunner queryRunner = new QueryRunner();

    /**
     * 执行增删改操作
     */
    public int baseUpdate(String sql, Object... params) {
        Connection conn = JDBCUtil.getConnection();
        try {
            return queryRunner.update(conn, sql, params);
        } catch (SQLException e) {
            throw new RuntimeException(e);
        } finally {
            JDBCUtil.close(conn);
        }
    }
}

```



```

/**
 * 查询单条记录
 */
public <T> T baseQueryOne(Class<T> clazz, String sql, Object... params)
{
    Connection conn = JDBCUtil.getConnection();
    try {
        return queryRunner.query(conn, sql, new BeanHandler<>(clazz),
params);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    } finally {
        JDBCUtil.close(conn);
    }
}

/**
 * 查询多条记录
 */
public <T> List<T> baseQueryList(Class<T> clazz, String sql, Object...
params) {
    Connection conn = JDBCUtil.getConnection();
    try {
        return queryRunner.query(conn, sql, new BeanListHandler<>
(clazz), params);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    } finally {
        JDBCUtil.close(conn);
    }
}
}

```

SysUserDaoImpl - 具体实现:

```

public class SysUserDaoImpl extends BaseDao implements SysUserDao {
    @Override
    public int addSysUser(SysUser sysUser) {
        String sql = "INSERT INTO sys_user (username, user_pwd) VALUES (?,
?)"
        return baseUpdate(sql, sysUser.getUsername(), sysUser.getUserPwd());
    }

    @Override
    public SysUser findByUsername(String username) {

```

```

        String sql = "SELECT uid, username, user_pwd as userPwd FROM
sys_user WHERE username = ?";
        return baseQueryOne(SysUser.class, sql, username);
    }
}

```

2.4 登录过滤器实现

```

@WebFilter({" /showSchedule.html", "/schedule/*"})
public class LoginFilter implements Filter {
    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse
servletResponse,
                        FilterChain filterChain) throws IOException,
ServletException {
        HttpServletRequest req = (HttpServletRequest) servletRequest;
        HttpServletResponse resp = (HttpServletResponse) servletResponse;

        HttpSession session = req.getSession();
        SysUser loginUser = (SysUser) session.getAttribute("loginUser");

        if (loginUser == null) {
            resp.sendRedirect(req.getContextPath() + "/login.html");
            return;
        }

        filterChain.doFilter(req, resp);
    }

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {}

    @Override
    public void destroy() {}
}

```

三、实验结果

3.1 数据库表结构

```

-- 用户表
CREATE TABLE sys_user (
    uid INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE,

```

```

        user_pwd VARCHAR(64) NOT NULL
    );

-- 日程表
CREATE TABLE sys_schedule (
    sid INT PRIMARY KEY AUTO_INCREMENT,
    uid INT NOT NULL,
    title VARCHAR(255),
    completed INT DEFAULT 0,
    CONSTRAINT fk_schedule_user FOREIGN KEY (uid) REFERENCES sys_user(uid)
ON DELETE CASCADE
);

```

3.2 完整的工作流程

用户访问流程

1. 用户打开浏览器访问 `http://localhost:8080/`
↓
2. 加载登录页面 (`login.html`)
 - ├ 显示用户名和密码输入框
 - └ 提供登录和注册链接
 ↓
3. 用户输入用户名和密码，点击登录
↓
4. 浏览器发送 POST 请求到 `/user/login?method=login`
↓
5. `BaseController.service()` 使用反射调用 `SysUserController.login()`
↓
6. `login()` 方法调用 `SysUserService.login()` 进行身份验证
 - ├ 查询数据库: `SELECT * FROM sys_user WHERE username = ?`
 - ├ 验证密码: `MD5(输入密码) == 数据库密码`
 - └ 返回 `SysUser` 对象或 `null`
 ↓
7. 验证成功
 - ├ 获取 `HttpSession`
 - ├ `session.setAttribute("loginUser", sysUser)`
 - └ 重定向到 `/showSchedule.html`
 ↓
8. 浏览器请求 `/showSchedule.html`
 - ├ `LoginFilter` 检查会话中的 `loginUser`
 - ├ `loginUser != null` → 继续执行
 - └ 返回日程管理页面
 ↓
9. 加载日程管理页面

- └ 显示欢迎信息 (Hello, 用户名)
- └ 显示现有日程列表
- └ 提供添加、编辑、删除日程功能

数据库操作流程

添加日程：

用户输入 → 发送 AJAX 请求 → SysScheduleController.add()

↓

SysScheduleService.addSchedule()

- └ 创建 SysSchedule 对象
- └ 调用 SysScheduleDaoImpl.addSchedule()
- └ SQL: INSERT INTO sys_schedule (uid, title, completed) VALUES (?, ?, ?)

↓

数据保存成功 → 返回 "新增成功" → 页面刷新日程列表

查询日程：

页面加载 → 发送 AJAX 请求 → SysScheduleController.list()

↓

SysScheduleService.findByUid(uid)

- └ 调用 SysScheduleDaoImpl.findByUid()
- └ SQL: SELECT sid, uid, title, completed FROM sys_schedule WHERE uid = ?

↓

DbUtils 自动将 ResultSet 映射为 SysSchedule 对象列表

↓

转换为 JSON 字符串 → 返回给前端

↓

JavaScript 解析 JSON → 动态渲染日程表

更新日程：

用户修改日程状态 → 发送 AJAX 请求 → SysScheduleController.update()

↓

SysScheduleService.updateSchedule()

- └ 调用 SysScheduleDaoImpl.updateSchedule()
- └ SQL: UPDATE sys_schedule SET title = ?, completed = ? WHERE sid = ?

↓

数据更新成功 → 返回 "更新成功" → 页面刷新日程列表

删除日程：

用户点击删除 → 发送 AJAX 请求 → SysScheduleController.delete()

↓

SysScheduleService.deleteSchedule()

- └ 调用 SysScheduleDaoImpl.deleteSchedule()
- └ SQL: DELETE FROM sys_schedule WHERE sid = ?

↓

数据删除成功 → 返回 "删除成功" → 页面刷新日程列表

3.3 系统防护机制

- 第一层：过滤器防护
 - └ @WebFilter({"showSchedule.html", "/schedule/*"})
检查会话中的 loginUser，未登录则重定向
- 第二层：控制器防护
 - └ add() 方法中再次检查会话中的 loginUser
- 第三层：业务逻辑防护
 - └ Service 层验证用户权限和数据合法性
- 第四层：数据库防护
 - └ 使用参数化查询，防止 SQL 注入
使用外键约束，防止数据不一致

3.4 技术亮点总结

技术点	实现	优势
反射路由	动态获取方法，通过 method 参数调用	代码简洁，易于扩展
通用 DAO	BaseDao 提供通用 CRUD 方法	代码复用，减少重复
会话管理	HttpSession 存储用户信息	跨请求保持状态
过滤器	LoginFilter 拦截未授权请求	统一身份认证
连接池	Druid 连接池管理数据库连接	性能优化，资源复用
MD5 加密	密码单向加密存储	保护用户隐私
参数化查询	使用 ? 占位符和参数数组	防止 SQL 注入
MVC 分层	Model、View、Controller 分离	架构清晰，易于维护

四、常见问题

无